2a. Claude and ChatGPT

2b. On my personal laptop.

2c. Here is the prompt I used:

Ground Rules:

- Code will be in C++ and able to be compiled and run on Linux servers
- Object-oriented design: classes and objects. NOT piled up standalone functions.
- Main ONLY contains coordinates, NOT the logic

Input:

- There will be 5 sample files that will be tested and are in the same directory as the program
- A filled cell will have a single digit 1-9
- An empty cell is indicated by an underscore _
- A file of the exact layout will be attached below for reference
- Make sure an error message occurs instead of a crash when handling a missing or malformed file

Algorithm and Rules:

- 9x9 grid made from nine 3x3
- Each digit 1-9 appears once per row, column, and block
- Brute-force depth-first search with recursive backtracking
- When an empty cell is found. For everyone, find the candidate digits (those that do not appear in the row, column, or block). If there are none, you'll backtrack. Otherwise, recursively try each candidate.
- FIND ALL SOLUTIONS; do not stop at the first puzzle. A puzzle can have zero, one, or several digits.

Output:

- For each puzzle, the file name, the original puzzle printed, finally all the solutions OR the exact text "No solution found"
- Say how a grid SHOULD be printed to be considered readable (each row on it's own line, potentially a block separator)
- Maybe (if desired) a count of solutions found

Quality Expectations:

- MEANINGFUL names and CLEAR structures

(I gave them the file of puzzle 1)

2d:

(Claude)

```
// Sudoku all-solutions solver: brute-force DFS with recursive backtracking.
// Build:  g++ -std=c++17 -O2 -Wall -Wextra -o sudoku_solver sudoku_solver.cpp
// Run:    ./sudoku_solver              (uses puzzle1.txt ... puzzle5.txt)
//         ./sudoku_solver a.txt b.txt  (uses the given files)

#include <array>
#include <cctype>
#include <fstream>
#include <iostream>
#include <sstream>
#include <stdexcept>
#include <string>
#include <vector>

// ---------------------------------------------------------------------------
// Thrown when a puzzle file is missing or does not follow the expected layout.
// ---------------------------------------------------------------------------
class PuzzleFileError : public std::runtime_error {
public:
    explicit PuzzleFileError(const std::string& message) : std::runtime_error(message) {}
};

// ---------------------------------------------------------------------------
// SudokuGrid: a 9x9 board. Knows the rules (row / column / block uniqueness)
// and how to print itself. Contains no solving strategy.
// ---------------------------------------------------------------------------
class SudokuGrid {
public:
    static constexpr int kSize = 9;
    static constexpr int kBlockSize = 3;
    static constexpr int kEmpty = 0;

    SudokuGrid() {
        for (auto& row : cells_) row.fill(kEmpty);
    }

    int  valueAt(int row, int col) const { return cells_[row][col]; }
    void setValue(int row, int col, int digit) { cells_[row][col] = digit; }
    void clearCell(int row, int col) { cells_[row][col] = kEmpty; }
    bool isEmptyAt(int row, int col) const { return cells_[row][col] == kEmpty; }
```

```
// Finds the first empty cell in row-major order. Returns false if the grid is full.
bool findFirstEmptyCell(int& row, int& col) const {
    for (row = 0; row < kSize; ++row)
        for (col = 0; col < kSize; ++col)
            if (isEmptyAt(row, col)) return true;
    return false;
}

// Digits 1-9 that do not appear in the cell's row, column, or 3x3 block.
std::vector<int> candidatesFor(int row, int col) const {
    std::array<bool, kSize + 1> used{};
    for (int i = 0; i < kSize; ++i) {
        used[cells_[row][i]] = true;
        used[cells_[i][col]] = true;
    }
    const int blockRow = (row / kBlockSize) * kBlockSize;
    const int blockCol = (col / kBlockSize) * kBlockSize;
    for (int r = 0; r < kBlockSize; ++r)
        for (int c = 0; c < kBlockSize; ++c)
            used[cells_[blockRow + r][blockCol + c]] = true;

    std::vector<int> candidates;
    for (int digit = 1; digit <= kSize; ++digit)
        if (!used[digit]) candidates.push_back(digit);
    return candidates;
}

// True if no digit repeats within any row, column, or block (empty cells ignored).
bool hasNoConflicts() const {
    for (int i = 0; i < kSize; ++i) {
        if (hasDuplicate(rowDigits(i)) || hasDuplicate(columnDigits(i)) ||
            hasDuplicate(blockDigits(i)))
            return false;
    }
    return true;
}

// Readable layout: one row per line, a space between digits, a "|" between
// blocks horizontally and a "------+-------+------" line between block rows.
// Empty cells are shown as "_".
std::string toReadableString() const {
    std::ostringstream out;
    for (int row = 0; row < kSize; ++row) {
        if (row > 0 && row % kBlockSize == 0) out << "------+-------+------\n";
```

```
            for (int col = 0; col < kSize; ++col) {
                if (col > 0) out << (col % kBlockSize == 0 ? " | " : " ");
                out << (isEmptyAt(row, col) ? '_' : static_cast<char>('0' + cells_[row][col]));
            }
            out << '\n';
        }
        return out.str();
    }

private:
    std::vector<int> rowDigits(int row) const {
        return std::vector<int>(cells_[row].begin(), cells_[row].end());
    }
    std::vector<int> columnDigits(int col) const {
        std::vector<int> digits;
        for (int r = 0; r < kSize; ++r) digits.push_back(cells_[r][col]);
        return digits;
    }
    std::vector<int> blockDigits(int blockIndex) const {
        std::vector<int> digits;
        const int startRow = (blockIndex / kBlockSize) * kBlockSize;
        const int startCol = (blockIndex % kBlockSize) * kBlockSize;
        for (int r = 0; r < kBlockSize; ++r)
            for (int c = 0; c < kBlockSize; ++c) digits.push_back(cells_[startRow + r][startCol + c]);
        return digits;
    }
    static bool hasDuplicate(const std::vector<int>& digits) {
        std::array<bool, kSize + 1> seen{};
        for (int d : digits) {
            if (d == kEmpty) continue;
            if (seen[d]) return true;
            seen[d] = true;
        }
        return false;
    }

    std::array<std::array<int, kSize>, kSize> cells_;
};

// ---------------------------------------------------------------------------
// PuzzleFileReader: turns a text file into a SudokuGrid, or throws
// PuzzleFileError with a clear message. Never crashes on bad input.
// Expected layout: 9 rows of 9 cells; each cell is a digit 1-9 or "_".
// Whitespace between cells is optional.
```

```
// ----------------------------------------------------------------------
class PuzzleFileReader {
public:
    SudokuGrid read(const std::string& fileName) const {
        std::ifstream input(fileName);
        if (!input) throw PuzzleFileError("Cannot open file (missing or unreadable).");

        const int cellCount = SudokuGrid::kSize * SudokuGrid::kSize;
        std::string symbols;
        char ch;
        while (input.get(ch)) {
            if (std::isspace(static_cast<unsigned char>(ch))) continue;
            if (!isValidSymbol(ch))
                throw PuzzleFileError(std::string("Unexpected character '") + ch +
                                      "'; only digits 1-9 and '_' are allowed.");
            symbols.push_back(ch);
            if (static_cast<int>(symbols.size()) > cellCount)
                throw PuzzleFileError("Too many cells; expected exactly 81.");
        }
        if (input.bad()) throw PuzzleFileError("Error while reading file.");
        if (static_cast<int>(symbols.size()) != cellCount)
            throw PuzzleFileError("Expected 81 cells but found " + std::to_string(symbols.size()) +
".");

        SudokuGrid grid;
        for (int i = 0; i < cellCount; ++i) {
            if (symbols[i] != '_')
                grid.setValue(i / SudokuGrid::kSize, i % SudokuGrid::kSize, symbols[i] - '0');
        }
        return grid;
    }

private:
    static bool isValidSymbol(char ch) { return ch == '_' || (ch >= '1' && ch <= '9'); }
};

// ----------------------------------------------------------------------
// SudokuSolver: recursive backtracking DFS that collects EVERY solution.
// ----------------------------------------------------------------------
class SudokuSolver {
public:
    // Returns all completed grids reachable from the starting puzzle.
    std::vector<SudokuGrid> findAllSolutions(const SudokuGrid& puzzle) {
        solutions_.clear();
```

```
        if (!puzzle.hasNoConflicts()) return solutions_;  // given clues already break the rules
        SudokuGrid working = puzzle;
        search(working);
        return solutions_;
    }

private:
    void search(SudokuGrid& grid) {
        int row = 0, col = 0;
        if (!grid.findFirstEmptyCell(row, col)) {  // no empty cells: a full solution
            solutions_.push_back(grid);
            return;
        }
        for (int digit : grid.candidatesFor(row, col)) {  // none -> loop skipped -> backtrack
            grid.setValue(row, col, digit);
            search(grid);
            grid.clearCell(row, col);
        }
    }

    std::vector<SudokuGrid> solutions_;
};

// ---------------------------------------------------------------------------
// PuzzleReporter: formats and prints the result for one puzzle.
// ---------------------------------------------------------------------------
class PuzzleReporter {
public:
    explicit PuzzleReporter(std::ostream& out) : out_(out) {}

    void printHeader(const std::string& fileName) const {
        out_ << "==============================\n"
             << "File: " << fileName << "\n"
             << "==============================\n";
    }

    void printOriginal(const SudokuGrid& puzzle) const {
        out_ << "Original puzzle:\n" << puzzle.toReadableString() << '\n';
    }

    void printSolutions(const std::vector<SudokuGrid>& solutions) const {
        if (solutions.empty()) {
            out_ << "No solution found\n\n";
            return;
```

```
        }
        for (std::size_t i = 0; i < solutions.size(); ++i) {
            out_ << "Solution " << (i + 1) << ":\n" << solutions[i].toReadableString() << '\n';
        }
        out_ << "Total solutions found: " << solutions.size() << "\n\n";
    }

    void printError(const std::string& fileName, const std::string& message) const {
        out_ << "==============================\n"
             << "File: " << fileName << "\n"
             << "==============================\n"
             << "ERROR: " << message << "\n\n";
    }

private:
    std::ostream& out_;
};

// ---------------------------------------------------------------------------
// PuzzleBatchRunner: coordinates reading, solving and reporting for each file.
// ---------------------------------------------------------------------------
class PuzzleBatchRunner {
public:
    explicit PuzzleBatchRunner(std::vector<std::string> fileNames)
        : fileNames_(std::move(fileNames)), reporter_(std::cout) {}

    void runAll() {
        for (const std::string& fileName : fileNames_) runOne(fileName);
    }

private:
    void runOne(const std::string& fileName) {
        try {
            SudokuGrid puzzle = reader_.read(fileName);
            reporter_.printHeader(fileName);
            reporter_.printOriginal(puzzle);
            reporter_.printSolutions(solver_.findAllSolutions(puzzle));
        } catch (const PuzzleFileError& error) {
            reporter_.printError(fileName, error.what());
        } catch (const std::exception& error) {
            reporter_.printError(fileName, std::string("Unexpected failure: ") + error.what());
        }
    }
```

```
    std::vector<std::string> fileNames_;
    PuzzleFileReader reader_;
    SudokuSolver solver_;
    PuzzleReporter reporter_;
};

// main only supplies the file names; all logic lives in the classes above.
int main(int argc, char* argv[]) {
    std::vector<std::string> fileNames;
    for (int i = 1; i < argc; ++i) fileNames.emplace_back(argv[i]);
    if (fileNames.empty())
        fileNames = {"puzzle1.txt", "puzzle2.txt", "puzzle3.txt", "puzzle4.txt", "puzzle5.txt"};

    PuzzleBatchRunner runner(fileNames);
    runner.runAll();
    return 0;
}
```

(ChatGPT)

```
#include <array>
#include <fstream>
#include <iostream>
#include <string>
#include <vector>

class SudokuGrid {
private:
    static constexpr int GRID_SIZE = 9;
    static constexpr int BLOCK_SIZE = 3;

    std::array<std::array<int, GRID_SIZE>, GRID_SIZE> cells{};

public:
    SudokuGrid() {
        for (auto& row : cells) {
            row.fill(0);
        }
    }

    bool setCell(int row, int column, int value) {
        if (row < 0 || row >= GRID_SIZE ||
            column < 0 || column >= GRID_SIZE ||
            value < 0 || value > 9) {
```

```
        return false;
    }

    cells[row][column] = value;
    return true;
}

int getCell(int row, int column) const {
    return cells[row][column];
}

bool isEmpty(int row, int column) const {
    return cells[row][column] == 0;
}

bool isValidPlacement(int row, int column, int value) const {
    // Check row.
    for (int currentColumn = 0; currentColumn < GRID_SIZE; ++currentColumn) {
        if (cells[row][currentColumn] == value) {
            return false;
        }
    }

    // Check column.
    for (int currentRow = 0; currentRow < GRID_SIZE; ++currentRow) {
        if (cells[currentRow][column] == value) {
            return false;
        }
    }

    // Check 3x3 block.
    int blockStartRow = (row / BLOCK_SIZE) * BLOCK_SIZE;
    int blockStartColumn = (column / BLOCK_SIZE) * BLOCK_SIZE;

    for (int currentRow = blockStartRow;
         currentRow < blockStartRow + BLOCK_SIZE;
         ++currentRow) {

        for (int currentColumn = blockStartColumn;
             currentColumn < blockStartColumn + BLOCK_SIZE;
             ++currentColumn) {

            if (cells[currentRow][currentColumn] == value) {
                return false;
```

```
            }
        }
    }

    return true;
}

std::vector<int> getCandidates(int row, int column) const {
    std::vector<int> candidates;

    for (int digit = 1; digit <= 9; ++digit) {
        if (isValidPlacement(row, column, digit)) {
            candidates.push_back(digit);
        }
    }

    return candidates;
}

bool findEmptyCell(int& row, int& column) const {
    for (row = 0; row < GRID_SIZE; ++row) {
        for (column = 0; column < GRID_SIZE; ++column) {
            if (isEmpty(row, column)) {
                return true;
            }
        }
    }

    return false;
}

bool isCompleteAndValid() const {
    for (int row = 0; row < GRID_SIZE; ++row) {
        for (int column = 0; column < GRID_SIZE; ++column) {
            int value = cells[row][column];

            if (value < 1 || value > 9) {
                return false;
            }
        }
    }

    // Verify every row.
    for (int row = 0; row < GRID_SIZE; ++row) {
```

```
    std::array<bool, 10> seen{};

    for (int column = 0; column < GRID_SIZE; ++column) {
        int value = cells[row][column];

        if (seen[value]) {
            return false;
        }

        seen[value] = true;
    }
}

// Verify every column.
for (int column = 0; column < GRID_SIZE; ++column) {
    std::array<bool, 10> seen{};

    for (int row = 0; row < GRID_SIZE; ++row) {
        int value = cells[row][column];

        if (seen[value]) {
            return false;
        }

        seen[value] = true;
    }
}

// Verify every 3x3 block.
for (int blockRow = 0; blockRow < GRID_SIZE; blockRow += BLOCK_SIZE) {
    for (int blockColumn = 0;
         blockColumn < GRID_SIZE;
         blockColumn += BLOCK_SIZE) {

        std::array<bool, 10> seen{};

        for (int row = blockRow;
             row < blockRow + BLOCK_SIZE;
             ++row) {

            for (int column = blockColumn;
                 column < blockColumn + BLOCK_SIZE;
                 ++column) {
```

```
                int value = cells[row][column];

                if (seen[value]) {
                    return false;
                }

                seen[value] = true;
            }
        }
    }

    return true;
  }

  const std::array<std::array<int, GRID_SIZE>, GRID_SIZE>& getCells() const {
    return cells;
  }
};


class PuzzleFileReader {
public:
  bool readPuzzle(const std::string& fileName,
                  SudokuGrid& puzzle,
                  std::string& errorMessage) const {

    std::ifstream inputFile(fileName);

    if (!inputFile.is_open()) {
      errorMessage = "Unable to open file.";
      return false;
    }

    std::string line;
    int row = 0;

    while (std::getline(inputFile, line)) {
      if (row >= 9) {
        errorMessage = "File contains more than 9 rows.";
        return false;
      }

      if (line.length() != 17) {
```

```
            errorMessage =
                "Each row must contain exactly 9 cells separated by spaces.";
            return false;
        }

        for (int column = 0; column < 9; ++column) {
            int characterPosition = column * 2;
            char character = line[characterPosition];

            if (column < 8 &&
                line[characterPosition + 1] != ' ') {

                errorMessage =
                    "Cells must be separated by single spaces.";
                return false;
            }

            if (character == '_') {
                puzzle.setCell(row, column, 0);
            }
            else if (character >= '1' && character <= '9') {
                puzzle.setCell(row, column, character - '0');
            }
            else {
                errorMessage =
                    "Invalid character found. Use digits 1-9 or '_'.";
                return false;
            }
        }

        ++row;
    }

    if (inputFile.bad()) {
        errorMessage = "An error occurred while reading the file.";
        return false;
    }

    if (row != 9) {
        errorMessage = "File must contain exactly 9 rows.";
        return false;
    }

    if (!hasValidInitialValues(puzzle)) {
```

```
            errorMessage =
                "The puzzle contains duplicate values in a row, column, or block.";
            return false;
        }

        return true;
    }

private:
    bool hasValidInitialValues(const SudokuGrid& puzzle) const {
        for (int row = 0; row < 9; ++row) {
            for (int column = 0; column < 9; ++column) {
                int value = puzzle.getCell(row, column);

                if (value == 0) {
                    continue;
                }

                // Temporarily check whether this value appears elsewhere.
                for (int otherColumn = 0; otherColumn < 9; ++otherColumn) {
                    if (otherColumn != column &&
                        puzzle.getCell(row, otherColumn) == value) {
                        return false;
                    }
                }

                for (int otherRow = 0; otherRow < 9; ++otherRow) {
                    if (otherRow != row &&
                        puzzle.getCell(otherRow, column) == value) {
                        return false;
                    }
                }

                int blockStartRow = (row / 3) * 3;
                int blockStartColumn = (column / 3) * 3;

                for (int blockRow = blockStartRow;
                     blockRow < blockStartRow + 3;
                     ++blockRow) {

                    for (int blockColumn = blockStartColumn;
                         blockColumn < blockStartColumn + 3;
                         ++blockColumn) {
```

```
                    if ((blockRow != row || blockColumn != column) &&
                        puzzle.getCell(blockRow, blockColumn) == value) {
                        return false;
                    }
                }
            }
        }

        return true;
    }
};


class SudokuSolver {
private:
    std::vector<SudokuGrid> solutions;

public:
    void solve(const SudokuGrid& puzzle) {
        solutions.clear();

        SudokuGrid workingPuzzle = puzzle;
        searchForSolutions(workingPuzzle);
    }

    const std::vector<SudokuGrid>& getSolutions() const {
        return solutions;
    }

private:
    void searchForSolutions(SudokuGrid& puzzle) {
        int row = 0;
        int column = 0;

        // No empty cells means a complete solution was found.
        if (!puzzle.findEmptyCell(row, column)) {
            solutions.push_back(puzzle);
            return;
        }

        // Find every candidate for this empty cell.
        std::vector<int> candidates =
            puzzle.getCandidates(row, column);
```

```
        // No candidates means this branch cannot produce a solution.
        if (candidates.empty()) {
            return;
        }

        // Try every candidate recursively.
        for (int candidate : candidates) {
            puzzle.setCell(row, column, candidate);

            searchForSolutions(puzzle);

            // Backtrack so the next candidate can be tested.
            puzzle.setCell(row, column, 0);
        }
    }
};


class SudokuPrinter {
public:
    void printPuzzle(const SudokuGrid& puzzle) const {
        printGrid(puzzle);
    }

    void printSolutions(const std::vector<SudokuGrid>& solutions) const {
        if (solutions.empty()) {
            std::cout << "No solution found\n";
            return;
        }

        std::cout << "Solutions found: "
                  << solutions.size()
                  << "\n\n";

        for (std::size_t solutionNumber = 0;
             solutionNumber < solutions.size();
             ++solutionNumber) {

            std::cout << "Solution " << solutionNumber + 1 << ":\n";
            printGrid(solutions[solutionNumber]);
            std::cout << '\n';
        }
    }
```

```
private:
    void printGrid(const SudokuGrid& grid) const {
        for (int row = 0; row < 9; ++row) {
            if (row == 3 || row == 6) {
                std::cout << "------+-------+------\n";
            }

            for (int column = 0; column < 9; ++column) {
                if (column == 3 || column == 6) {
                    std::cout << "| ";
                }

                int value = grid.getCell(row, column);

                if (value == 0) {
                    std::cout << "_";
                }
                else {
                    std::cout << value;
                }

                if (column != 8) {
                    std::cout << ' ';
                }
            }

            std::cout << '\n';
        }
    }
};


class SudokuProgram {
private:
    PuzzleFileReader fileReader;
    SudokuPrinter printer;

public:
    void processFiles(const std::vector<std::string>& fileNames) const {
        for (const std::string& fileName : fileNames) {
            processSingleFile(fileName);
        }
    }
```

```
private:
    void processSingleFile(const std::string& fileName) const {
        SudokuGrid puzzle;
        std::string errorMessage;

        std::cout << "========================================\n";
        std::cout << "Puzzle: " << fileName << "\n";
        std::cout << "========================================\n";

        if (!fileReader.readPuzzle(fileName, puzzle, errorMessage)) {
            std::cout << "Error: " << errorMessage << "\n\n";
            return;
        }

        std::cout << "Original puzzle:\n";
        printer.printPuzzle(puzzle);
        std::cout << "\n";

        SudokuSolver solver;
        solver.solve(puzzle);

        printer.printSolutions(solver.getSolutions());
        std::cout << '\n';
    }
};


int main() {
    // The main function only identifies the input files and starts the program.
    std::vector<std::string> puzzleFiles = {
        "puzzle1.txt",
        "puzzle2.txt",
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
    };

    SudokuProgram program;
    program.processFiles(puzzleFiles);

    return 0;
}
```

2e.

Correctness: Both Gen. AI programs were able to compile without any errors; however, ChatGPT's program failed, with all 5 puzzles being rejected with the message: "Each row must contain exactly 9 cells separated by spaces." This is because the file reader requires every line to be exactly 17 characters long; if it detects any difference, such as spacing or line endings, from what is expected, it immediately fails. Thus, because the program could never read the files, ChatGPT could never solve the puzzles. On the other hand, Claude's program solved all 5 puzzles. This was shown by its failure to find a solution for puzzle 4; puzzles 1, 2, and 3 had solutions, with one matching the solution provided on Canvas, and for puzzle 5 it found two solutions. Additionally, it's file reader skipped all whitespace and read 81 symbols, which resulted in its capability to handle the puzzle files without any trouble

Execution Time: Both programs seemed to use the same recursive backtracking, always picking the first empty cell. In the worst case, this is O(9^k), where k is the number of empty cells, because each empty cell has up to 9 candidates. In practice, the search is much faster because a digit is tried only if it does not already appear in the cell's row, column, or block, allowing early elimination of most branches. Furthermore, the work done at each step is O(n^2) for both programs, Where N is 9, due to rescanning the grid for the next empty cell. In both cases, GenAI's approach is similar: Claude's candidate lookup makes a single Pass over the row/column in blocks; on the other hand, ChatGPT makes 9 separate validity checks. In my opinion, Claude takes fewer steps to get the results. However, an issue arises when comparing the two's execution time because ChatGPT

Space Complexity: The two Gen. AI programs use nearly the same amount of memory; in fact, recursion goes at most K levels deep, with each level holding a small vector of at most 9 candidate digits; therefore, the call stack uses O(k * n). The working grid has one copy of size O(n^2). Additionally, both programs stored every solution they found as a full copy of the grid.

Maintainability: The programs divided the work into separate classes for the grids, file readers, solver, and output. Additionally, they kept main simple. Furthermore, ChatGPT's program writes all four, includes 2 methods that are never called, and repeats its duplicate-checking logic in more than one place. On the other hand, Claude's program uses constraints consistently, has no unused methods, and reports file problems with exceptions, separating error handling from the solving code. Its main issues are the lack of comments and the fact that the code that prints the file header appears in two places.

2f.

For this assignment, I chose Claude. The main reason is that it can process all 5 puzzles and output them correctly, handle unique, no, and two-solution cases, and match the solution in the first PDF.

2g.

Correctness: Nothing was changed in terms of correctness due to Claude being able to fully process all 5 puzzles, with the correct output being able to handle a no-solution, two-solution case, and handle unique cases.

Execution Time: Nothing was changed for execution time due to no huge errors being seen or delays.

Space Complexity: Nothing was changed due to both programs using the same logic. Neither was faster than the other.

Maintainability: I merged the repeated header-printing code, and justice made it a single method and added comments, including a prologue.